Quantization-Aware Adaptive LoRA

Yinhao Zhao, Yihui Li

2026/04/28

LoRA: Low-Rank Adaptation

Full fine-tuning: update all parameters W from pre-trained $W^{(0)}$.

$$\min_{W} \ \mathcal{L}(W) := \frac{1}{N} \sum_{i=1}^{N} \ell(f(x_i; W), y_i)$$

LoRA idea: freeze $W^{(0)} \in \mathbb{R}^{d \times k}$, train a low-rank update $W = W^{(0)} + \Delta W = W^{(0)} + BA$.

$$\min_{A,B} \ \mathcal{L}\Big(W^{(0)} + BA\Big), \qquad B \in \mathbb{R}^{d \times r}, \quad A \in \mathbb{R}^{r \times k}, \quad r \ll \min(d, k)$$

- d: output dimension; k: input dimension; r: adapter rank.
- Trainable parameters: $r(d + k)$ instead of dk.

QLoRA: LoRA with Quantized Weights

Quantization: store the weights by low-bit codes $\bar{W}^{(0)}$ with the scale $s^{(0)}$ for dequantization.

$$\bar{W}^{(0)} = Q(W^{(0)}) \approx \left\lfloor \frac{W^{(0)}}{s^{(0)}} \right\rceil$$

QLoRA idea: freeze the low-bit pretrained weights $\bar{W}^{(0)}$. During computation, dynamically dequantize them to floating-point, and then train LoRA.

$$\widehat{W}^{(0)} = \mathrm{DeQuant}\left(\bar{W}^{(0)}\right) \in \mathbb{R}^{d\times k}$$

$$\min_{A,B} \mathcal{L}\left(\widehat{W}^{(0)} + BA\right), \qquad B \in \mathbb{R}^{d\times r}, \quad A \in \mathbb{R}^{r\times k}, \quad r \ll \min(d,k)$$

- Lower memory (e.g. quantizing the weights from FP32 to 4-bit can shrink by $8\times$).
- Introduces quantization error $E_q = W^{(0)} - \widehat{W}^{(0)}$.

AdaLoRA: LoRA with Adaptive Budget Allocation

Motivation: different layers may need different adaptation capacity.

SVD-based Adaptation: replace the fixed-rank BA with $P\Lambda Q$.

$$\Delta W = P\Lambda Q, \qquad P \in \mathbb{R}^{d\times\tilde{r}}, \quad \Lambda = \mathrm{diag}(\lambda_1, \ldots, \lambda_{\tilde{r}}), \quad Q \in \mathbb{R}^{\tilde{r}\times k}$$

AdaLoRA idea: dynamically allocate the rank budget per layer by adding rank and orthogonal regularizations.

$$\min_{P,\Lambda,Q} \mathcal{L}\Big(W^{(0)} + P\Lambda Q\Big) + \alpha\,\Phi(\Lambda) + \beta\,\mathcal{R}(P,Q)$$

- $\Phi(\Lambda)$: sparsifies singular values $\Rightarrow$ low effective rank.
- $\mathcal{R}(P,Q) = \|P^TP - I\|_F^2 + \|QQ^T - I\|_F^2 \Rightarrow$ enforces orthogonality.
- Allocate more rank to important layers under a fixed budget.

Our idea: Quantization-Aware Adaptive LoRA

Observation: QLoRA introduces layer-wise quantization error $E_q = W^{(0)} - \widehat{W}^{(0)}$, whose impact varies significantly across layers.

Our idea: explicitly utilize E_q to guide the adaptive rank allocation (instead of relying solely on SVD regularizations).

- **Quantization effect**: Explore how E_q specifically affects the adaptation process.
- **Error-driven allocation**: Dynamically allocate higher rank to layers with larger E_q to compensate for the quantization precision loss.

Experimental Plan

Goal: analyze the effect of quantization and evaluate quantization-aware rank allocation.

Stage 1: Quantization effect. Measure performance drop and layer-wise perturbation after quantizing the frozen backbone.

$$W^{(0)} \to \widehat{W}^{(0)}, \qquad E_q^{(\ell)} = W^{(0,\ell)} - \widehat{W}^{(0,\ell)}$$

Stage 2: Error-driven allocation. Use quantization error as the rank-allocation signal.

$$e_\ell = \frac{\|E_q^{(\ell)}\|_F}{\|W^{(0,\ell)}\|_F}, \qquad r_\ell = \mathrm{Allocate}(e_\ell), \qquad \sum_\ell r_\ell \le R$$

Comparison: LoRA, QLoRA with fixed rank, AdaLoRA, and our method.

Expected Outcome and Future Work

Expected outcome

- Analyze the layer-wise effect of quantization in QLoRA.
- Design an error-driven rank allocation strategy.
- Compare fixed-rank, adaptive-rank, and quantization-aware methods under the same budget.

Future work

- Combine quantization error with layer sensitivity for better rank allocation.
- Explore other quantization-related training methods, such as mixed-precision SGD.

Thank you!